

Lab 8: Memory Ordering Lab

In this lab, you will use the SUSTemu RISC-V simulator to observe how the TSO (Total Store Order) memory model causes observable concurrency bugs in dual-core programs, and how **fence** instructions restore the ordering guarantees needed for correct synchronization. By the end of this lab, you will be able to:

1. Demonstrate a Store-Load reordering violation using the classic Dekker litmus test
2. Explain how a per-core store buffer causes lost updates in a non-atomic counter
3. Identify why Peterson's mutex fails under TSO and add the minimal fences to fix it

Default simulator configuration:

- Two OOO cores: **--dual --ooo --bpred -b**
- Each core has a per-core store buffer (**STORE_BUF_SIZE = 16**); committed stores drain one entry per cycle to the shared L2 cache
- TSO model: **LOAD** can pass an older **STORE** to a different address (Store-Load reordering); Store-Store order is preserved
- Shared memory region: **MEM_BASE = 0x80200000**; MMIO UART at **0xa00003f8**

Each benchmark directory provides two make targets:

```
make run          # run without fence between store and load
make run-fence    # run with fence rw,rw inserted
```

Note: after modifying any file under **include/**, always run **make clean && make** from the repository root before running lab benchmarks.

Step 1 — Store-Load Reordering

Background

`order_lab/Q1/` contains a classic **Dekker litmus test**. Two cores execute the following sequence in parallel for 200 trials:

```
Initially:  x = 0,  y = 0

Core 0:    x = 1;  [fence?]  r1 = y
Core 1:    y = 1;  [fence?]  r2 = x
```

Under **Sequential Consistency (SC)**: at least one store must be globally visible before the other core's load, so the outcome `r1 = 0 AND r2 = 0` is **impossible**.

Under **TSO**: both stores can sit in each core's store buffer while both loads read stale values from the shared cache. The outcome `r1 = 0 AND r2 = 0` is **observable** — this is a TSO violation.

A **rendezvous barrier** (using monotonic sequence numbers and `fence rw,rw`) synchronizes the two cores between trials so that each trial starts from a clean state. The barrier itself is unrelated to the litmus sequence being tested.

```
cd order_lab/Q1
make run          # no fence between store and load
make run-fence    # fence rw,rw between store and load
```

Question 1: Run `make run`. Record the number and percentage of violations from the output. Then run with `make run-inorder` (edit `FLAGS` in the Makefile to use `--inorder --bpred -b` instead of `---ooo`). Explain why the in-order pipeline produces zero violations, and which property of the OOO + TSO combination makes violations possible.

Question 2: Run `make run-fence`. Explain why inserting `fence rw,rw` between the store and the load eliminates the `r1 = 0 AND r2 = 0` outcome. Draw a timeline showing the happens-before relationship established by the fence for both cores.

Step 2 — Lost Updates

Background

`order_lab/Q2/` contains two cores each performing 1000 increments of a shared counter. Each increment is a non-atomic **read-modify-write** sequence:

```
tmp = *counter;    // LOAD → may read from shared cache, missing the other
core's store buffer
tmp = tmp + 1;
*counter = tmp;    // STORE → enters this core's store buffer
```

Because the store enters the store buffer and is not immediately visible to the other core, both cores can read the same value `k`, both compute `k+1`, and both write `k+1` back — one increment is lost. With 2×1000 iterations and a fully-loaded store buffer, nearly every increment can be a lost update.

```
cd order_lab/Q2
make run
```

Question 3: Run `make run`. Record the actual counter value and the expected value (2000). Calculate the number of lost updates. Explain why the loss count is close to `N_ITER` (1000) rather than a small fraction, using the store buffer drain latency to support your argument.

Step 3 — Peterson Mutual Exclusion Lock

Background

`order_lab/Q3/` implements **Peterson's algorithm** — a classic software mutex that requires no atomic instructions and is provably correct under SC:

```
lock(i):
    want[i] = 1;          // "I want to enter the critical section"
    turn    = 1 - i;      // "You go first"
    while (want[1-i] && turn == 1-i) {} // spin-wait

unlock(i):
    want[i] = 0;
```

Under TSO, `want[i] = 1` enters the store buffer and is **not immediately visible** to the other core. Both cores may observe `want[j] == 0` (stale cache value) and enter the critical section simultaneously — the mutex fails.

The fix is to add `fence rw, rw` after `want[i] = 1` (to drain the store buffer before reading `want[j]`) and after `turn = 1-i` (to prevent the turn store from being reordered past the load of `want[j]`). The `peterson.c` source contains two `TODO` markers indicating where the fences belong:

```
static void lock(int i) {
    int j = 1 - i;
    want[i] = 1;
    /* TODO (Q5): add fence rw, rw here */
    *turn = j;
    /* TODO (Q5): add fence rw, rw here */
    while (want[j] && *turn == j) {}
}
```

Both cores protect a shared counter with this lock, each incrementing it `N_ITER` times. The expected result is `2 * N_ITER` if the mutex is correct.

```
cd order_lab/Q3
make run          # USE_FENCE=0: lock is broken under TSO
make run-fence    # USE_FENCE=1: lock is correct
```

Question 4: Run `make run (USE_FENCE=0)`. Record the actual counter value versus the expected value. Explain which step of `lock()` fails under TSO. Draw a timeline showing how both cores simultaneously pass the spin-wait and enter the critical section.

Question 5: Open `order_lab/Q3/peterson.c`, find the two `TODO` comments inside `lock()`, and add `__asm__ volatile("fence rw, rw")` at each location. Run `make run-fence` to verify the counter equals `N_ITER` (all updates preserved). Explain why **two** fences are required — what memory reordering does each one prevent, and why a single fence is insufficient.